

Группа	Z3342	К работе допущен	_____
Студент	Погорелко В. А.	Работа выполнена	_____
Преподаватель	Маслов И. Д.	Отчёт принят	_____

Рабочий протокол и отчёт по лабораторной работе №1

[C++ & UNIX]: C++ BUILD / IF / LOOP, PYTHON

1. Цель работы

Познакомить студента с принципами компиляции исходного кода. Составить программу с использованием циклов, условий и функций. Сравнить быстродействие между C++ и Python. Ознакомление с типами данных.

2. Задачи

1. (C++ EXPRESSION) Создать и скомпилировать программу на C++
2. (PYTHON EXPRESSION) Создать и скомпилировать программу на Python 3
3. (SAVE) Результат всех вышеперечисленных шагов сохранить в репозиторий (+ отчет по данной ЛР в папку doc)

3. Решение

1. Создать и скомпилировать программу на C++

Для начала переместимся в директорию lab_02/src и создадим файл counter.cpp

Запрос:

```
boris@Boris:/$ cd /usr/local/cpp_linux_labs/lab_02/src
boris@Boris:/usr/local/cpp_linux_labs/lab_02/src$ nano \
\ counter.cpp
```

Далее пишем программу на языке C++

counter.cpp

```
#include <iostream>
#include <chrono>

using namespace std;

float iteratedFunction(float x) {
    return x*x - x*x + 4*x - 5*x + x + x;
}

int main() {
    char repeat;

    do {
        int n;
        cout << "Type a number of iterations: ";
        cin >> n;
        cout << endl;

        if (cin.fail()) {

            cout << "Invalid value" << endl;
            return 1;
        }

        float x = 0.13;
        float accum = 0.0; // accumulating result

        auto start = chrono::high_resolution_clock::now();

        for (int i = 0; i < n; i++) {
            accum += iteratedFunction(x);
        }

        auto end = chrono::high_resolution_clock::now();
        chrono::duration<double> delta = end - start;
        cout << "Time for " << n \
        \ << " iterations: " << delta.count() << " seconds";
        cout << endl;

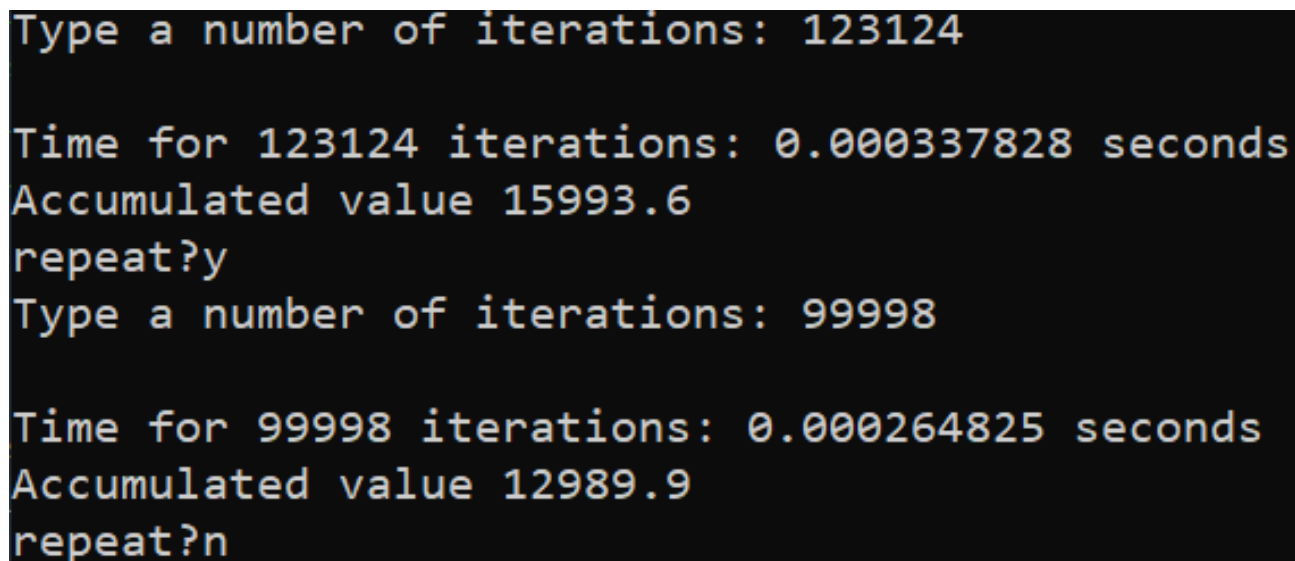
        cerr << "Accumulated value " << accum << endl;
        cout << "repeat?";
        cin >> repeat;
    } while (repeat == 'y');

    return 0;
}
```

Далее компилируем программу в папку /build и проверяем работу

Запрос:

```
boris@Boris:/usr/local/cpp_linux_labs/lab_02$ g++ \
\ src/counter.cpp -o build/counter
boris@Boris:/usr/local/cpp_linux_labs/lab_02$ cd /build
boris@Boris:/usr/local/cpp_linux_labs/lab_02/build$ ./counter
```



```
Type a number of iterations: 123124
Time for 123124 iterations: 0.000337828 seconds
Accumulated value 15993.6
repeat?y
Type a number of iterations: 99998
Time for 99998 iterations: 0.000264825 seconds
Accumulated value 12989.9
repeat?n
```

Рис. 1: Пример работы программы C++

2. Создать и скомпилировать программу на Python 3

В привычном понимании скомпилировать напрямую Python-код нельзя, он исполняется программой Python.exe, но байт-код получить можно. Так же в папке /src создадим файл python_counter.py

Запрос:

```
boris@Boris:/usr/local/cpp_linux_labs/lab_02/src$ cd /usr/local/cpp_linux_labs/lab_02/src
boris@Boris:/usr/local/cpp_linux_labs/lab_02/src$ nano \
\ python_counter.py
```

Внутри пишем python-код

python_counter.py

```
import time
import sys

def iterated_function(x):
    return x*x - x*x + 4*x - 5*x + x + x

def main():
    x = 0.13
    while True:
        try:
            n = int(input("Number of iterations: "))
        except ValueError:
            print("Invalid value")
            sys.exit(1)
        start = time.perf_counter()
        accum = 0.0
        for _ in range(n):
            accum += iterated_function(x)
        end = time.perf_counter()
        print(f"Time for {n} iterations: {end - start:.9f} sec")
        if input("Repeat? ") != 'y':
            break

if __name__ == "__main__":
    main()
```

Запуск можно производить сразу же после сохранения файла, используя Python:

Запрос:

```
boris@Boris:/usr/local/cpp_linux_labs/lab_02/src$ python3 \
\ python_counter.py
```

Скорость работы естественно медленнее, чем при выполнении C++ программы. Теперь нужно (по заданию) скомпилировать код, в Python можно выполнить сборку байт-кода программы для дальнейшего исполнения, сделать это можно с помощью запросов. Результат переместим в папку /build.

```
Number of iterations: 123124
Time for 123124 iterations: 0.011194922 sec
Repeat? (y/n): y
Number of iterations: 99998
Time for 99998 iterations: 0.009428507 sec
Repeat? (y/n): n
```

Рис. 2: Пример работы программы python

Запрос:

```
boris@Boris:/usr/local/cpp_linux_labs/lab_02$ python3 \
\ -m py_compile src/python_counter.py
boris@Boris:/usr/local/cpp_linux_labs/lab_02$ ls -l src
total 12
drwxr-xr-x 2 boris boris 4096 Apr 11 00:34 __pycache__
-rw-r--r-- 1 boris boris  757 Apr 10 23:37 counter.cpp
-rw-r--r-- 1 boris boris  690 Apr 10 23:43 python_counter.py
boris@Boris:/usr/local/cpp_linux_labs/lab_02$ mv \
\ src/__pycache__ build/__pycache__
```